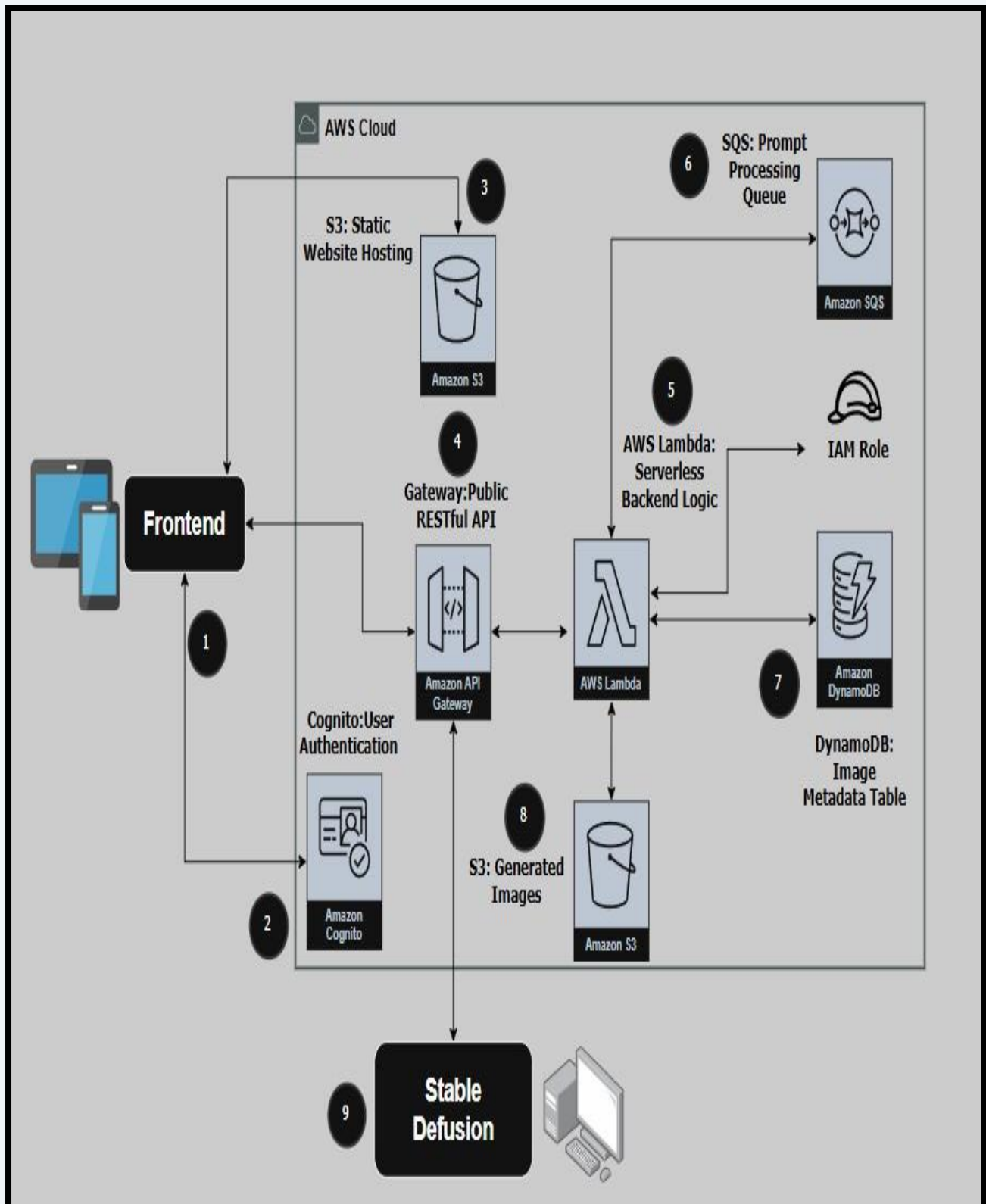


DefusionCraft: AI-Powered Prompt-to-Image Generation Platform on AWS

This reference architecture outlines *DefusionCraft*, a serverless platform on AWS for generating AI-powered images from user prompts. It combines services like Amazon S3, Lambda, API Gateway, and Cognito with local inference via Stable Diffusion. The architecture enables secure prompt submission, asynchronous processing, and dynamic image delivery—optimized for scalability, simplicity, and creativity.



1. **The frontend web application** is hosted on Amazon S3 and serves as the primary interface for users. It allows authenticated users to submit prompts through an AI chat assistant, view generated images in a personal gallery, and—if assigned to the Admin group—access moderation tools via a separate admin page. The interface communicates with backend APIs over HTTPS using JWT-based authorization.
2. **Amazon Cognito** manages user authentication and identity across the platform. Users log in through **Cognito's Hosted UI** and are issued **secure JWT tokens** upon successful authentication. These tokens are attached to all client-side API requests and are validated by backend services. **Cognito user pools** are configured with **group-based ("Admins", "Users")** access control to distinguish between regular users and admin users, enabling role-based feature access within the web application.
3. **Amazon S3** hosts the static assets of the DefusionCraft web application, including **HTML, JavaScript, and styling powered by Tailwind CSS**. These assets are delivered to users' browsers, providing a fast, responsive, and modern user experience.
4. **Amazon API Gateway** acts as the secure entry point for all client-initiated operations. It exposes **RESTful endpoints** that receive user requests such as prompt submission, image status checks, and image deletion. Each request must include a valid JWT token issued by Amazon Cognito, which is verified by backend Lambda functions. CORS is configured to allow safe cross-origin access from the web frontend.
5. **AWS Lambda** implements all backend logic required for DefusionCraft's operations. Functions are triggered by API Gateway and handle tasks such as **submitting prompts** to Amazon SQS, **storing and retrieving chat messages via the Gemini (Google AI) API**, and managing image-related actions including **download, preview, deletion, and prompt editing**. In addition, Lambda functions process **image uploads from the local generation worker** to Amazon S3, and **update the associated metadata** in Amazon DynamoDB. All functions are stateless and operate under the default LabRole IAM permissions provided by the AWS Academy environment. Wherever possible, access control is enforced at the application level using Cognito group validation.
6. **Amazon SQS** provides a decoupled, asynchronous messaging layer between the frontend and the image generation worker. Lambda functions enqueue user-submitted prompts into the queue, which are later consumed by a local worker that generates images using Stable Diffusion. This architecture isolates frontend responsiveness from processing time, improves fault tolerance, and enables reliable scaling of prompt handling.
7. **Amazon DynamoDB** stores metadata associated with each generated image, including the **prompt text, user identity, image status, timestamp, and the S3 storage path**. The table uses **imageId** as the **primary key** and includes a **global secondary index (GSI) on userSub** to enable fast retrieval of all images generated by a specific user. This setup supports scalable, low-latency access to image metadata for both regular users and admins.

8. **A separate Amazon S3 bucket** stores the final images generated by the local worker. Each image is uploaded with a **unique path that includes the user's identity and timestamp**, ensuring logical separation and traceability. All files are encrypted at rest using AWS-managed keys (**SSE-S3**) and are accessible only through authenticated API calls. Users can view, download, or delete their images via the web interface.
9. **A local Stable Diffusion worker** is implemented as a Python-based executable (**.exe**) file that runs continuously on a personal machine. It polls the Amazon SQS queue every 10 seconds for new prompt requests. Upon receiving a prompt, it generates an image using the **Hugging Face diffusers library**, uploads the image to Amazon S3, and updates the corresponding metadata in Amazon DynamoDB. This local processing approach enables cost-effective AI inference without the need for cloud-based GPU infrastructure.